Work Sample:
Onboarding Data and Code Migration White Paper

CONTENTS

CHAPTER 1 – INTRODUCTION **1**
WHY POSTGRESQL 2
REQUIREMENTS 3
DATA AND CODE MIGRATION OVERVIEW 4
CHAPTER 2 – GETTING STARTED WITH DATA MIGRATION **5**
Data Migration Tools and Services 6
AWS Schema Conversion Tool 6
AWS Database Migration Service 7
Migration Types 8
Full Data Migration 8
Incremental Data Migration 9
Pre-Live 11
Migration from VSG Hosting 11
Migration from On-Premises 12
Go Live 13
Migration from VSG Hosting 13
Migration from On-Premises 13
Report Generation 14
CHAPTER 3 – CODE MIGRATION **15**
CODE MIGRATION - ORACLE TO POSTGRES DATABASE 16
Prerequisites for Clients 16
How the Oracle to Postgres Code Conversion Process Works 17
Postgres Conversion Tool 17
CODE MIGRATION – JAVA TO POSTGRES DATABASE 19

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Chapter 1 – Introduction

has two types of clients starting their projects in Product.

1. New clients starting their implementation in Product
2. Existing clients who are on the premises or VSGHosting moving to Product

This guide is meant to address clients who fall under category 2. All prior V3 implementations were on Oracle and Weblogic and, as part of Product onboarding for existing clients, the very first step is to migrate client data from Oracle RDMBS to PostgreSQL RDBMS. The next step is to migrate all client configuration code (artifacts like batch, query, reports etc.,) from Oracle to PostgreSQL. This document outlines these steps in detail.

This document also provides a detailed overview of the Product onboarding process purely from Oracle to PostgreSQL migration of Data and code. Upgrading to the latest Product version, "post migration" is covered in a separate guide, *Upgrading to Product*. recommends that you read both the guides to get a complete understanding of the Product onboarding process.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Why PostgreSQL

PostgreSQL is a mature, open-source RDBMS and, it is widely used for a variety of applications worldwide. PostgreSQL comes very close to what Oracle offers in terms of database and security features, and, more importantly, AWS Aurora PostgreSQL includes features and benefits that are superior to what Oracle offers. AWS provides inbuilt, at rest encryption of data and AWS Aurora architecture provides high failover as it maintains 16 read replicas in different availability zones. Given all of the PostgreSQL features, bundled with AWS Aurora features, it is a natural choice for Product to use PostgreSQL.

The Product application (Java code) is designed to support both Oracle and PostgreSQL RDBMS. As part of the Product onboarding process, every client must go through a code migration process. This process is partly automated (60%), the remaining tasks require a manual effort.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Requirements

- Existing Product (V9 or lower versions) client using Oracle RDMBS moving to Product (V10 or later versions).
- Client's environment must reside in AWS Aurora RDS.
- AWS Data Migration Service and its related tools, such as SCT.
- The client must maintain a temporary Oracle RDS as a backup to hold data until the migration processes are complete.
- The implementation team must notify the client as to which Product Product version they will be migrated or upgraded.

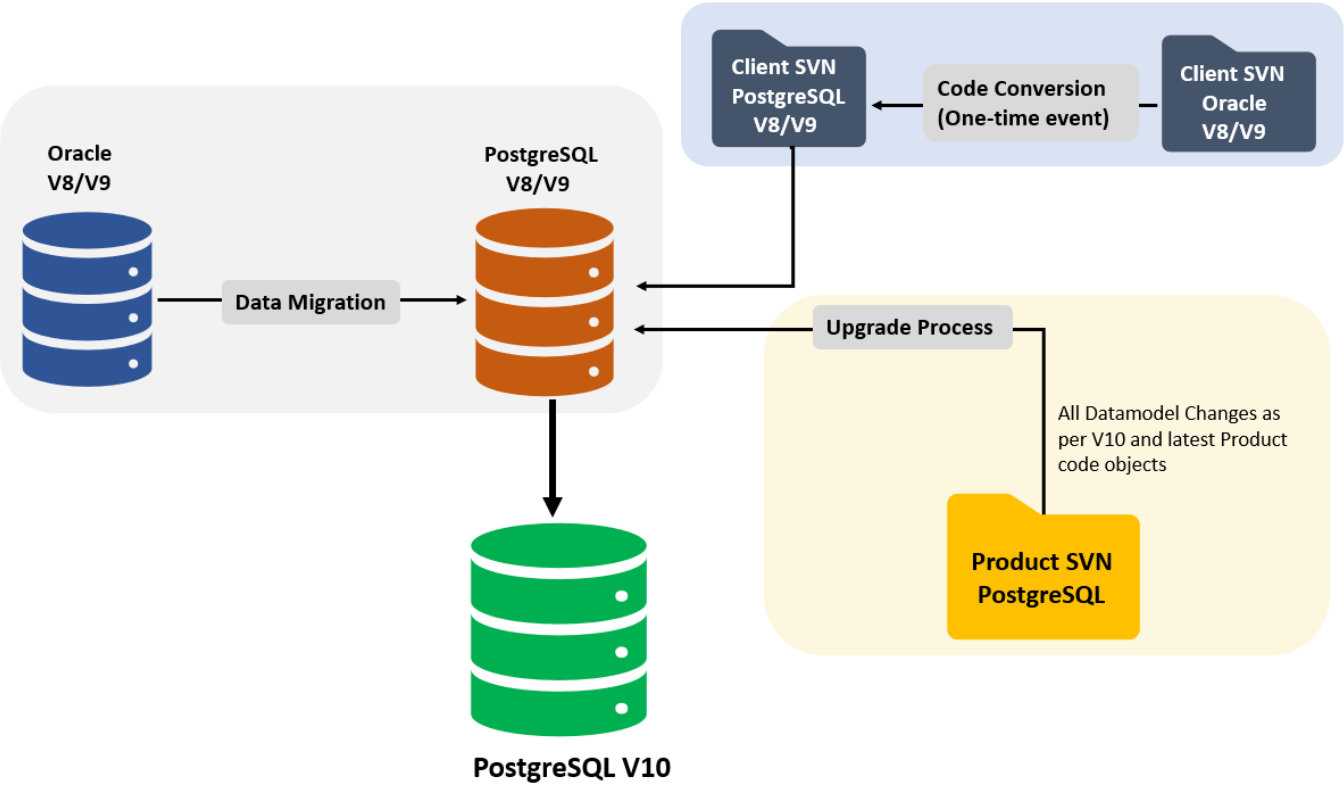
© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Data and Code Migration Overview

a) The client's data is migrated from Oracle to PostgreSQL.

b) The PostgreSQL database is upgraded to the latest Product version using the in-house Upgrade tool.

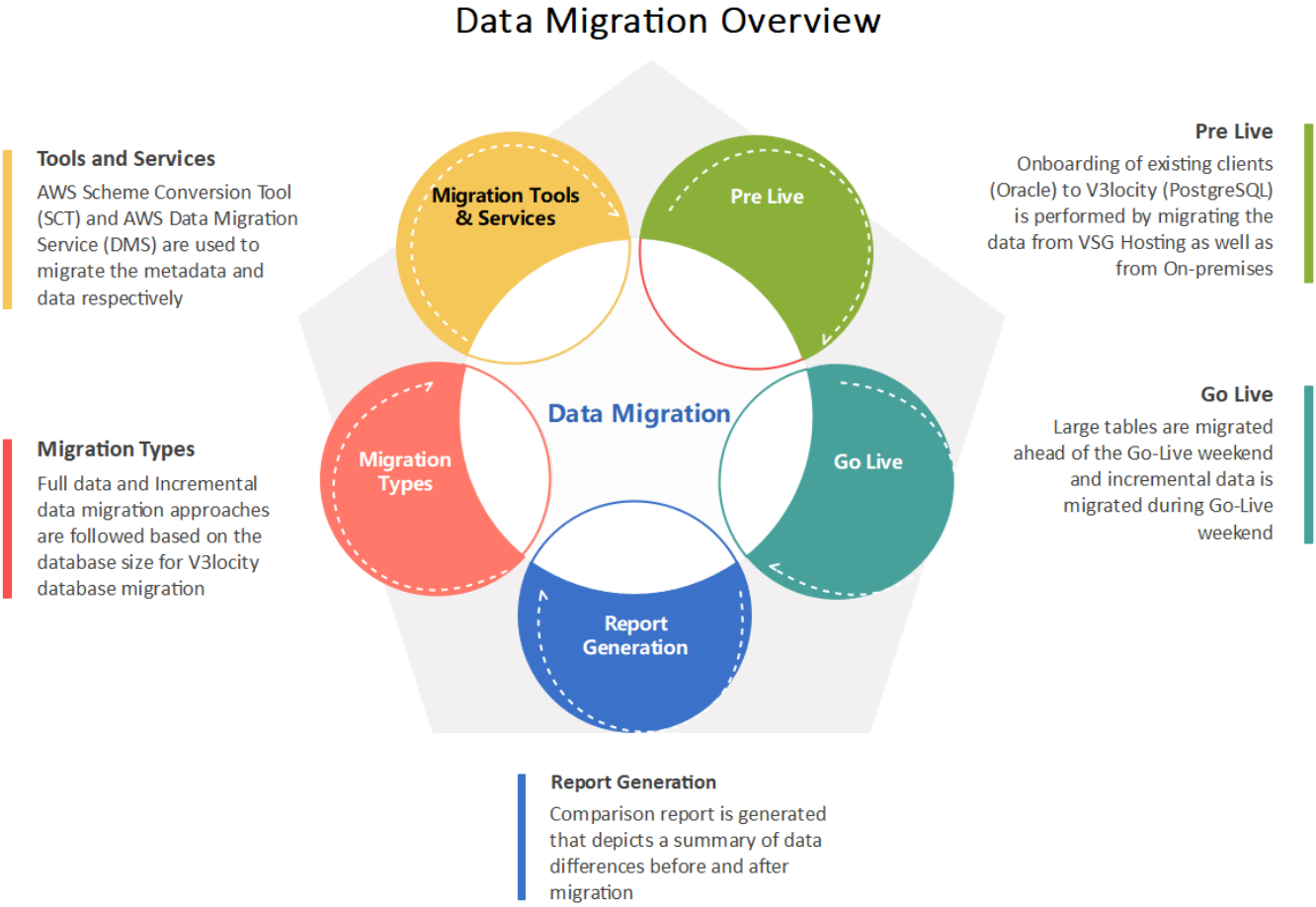
c) Client code objects that are migrated from Oracle to PostgreSQL are compiled against the client PostgreSQL database.



© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Chapter 2 – Getting Started with Data Migration

The Data migration process is performed to onboard clients into Product by migrating the data from Oracle to PostgreSQL database. Several factors, such as database size and, data center location of the source data, are considered to accomplish a seamless data transition with minimal downtime.



The following sections explain the Data Migration process for Product onboarding in detail:

[Data Migration Tool and Services]

[Migration Types]

[Pre Live]

[Go Live]

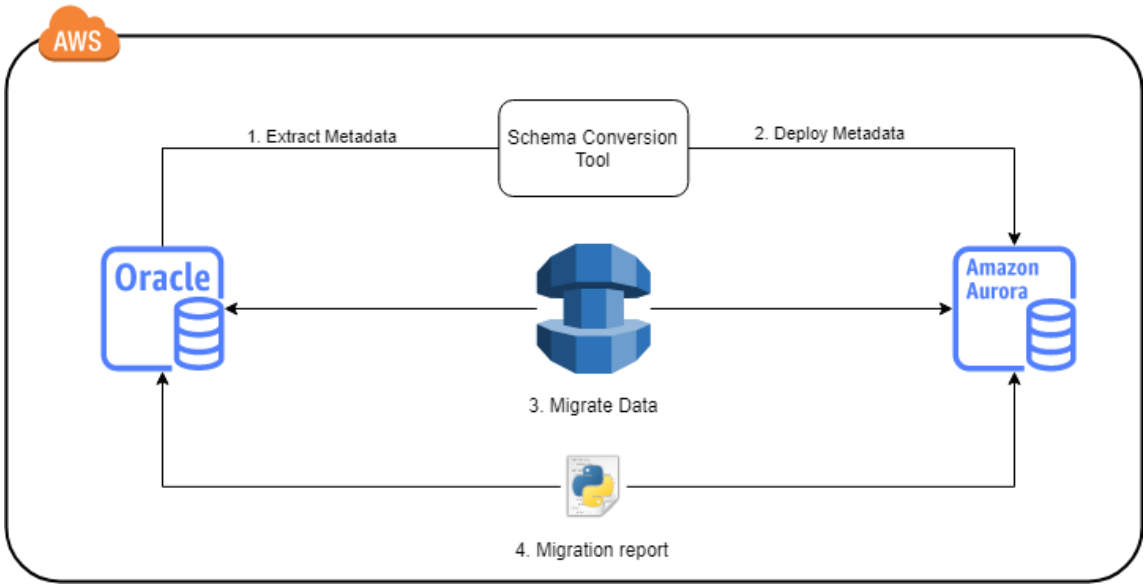
[Report Generation]

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Data Migration Tools and Services

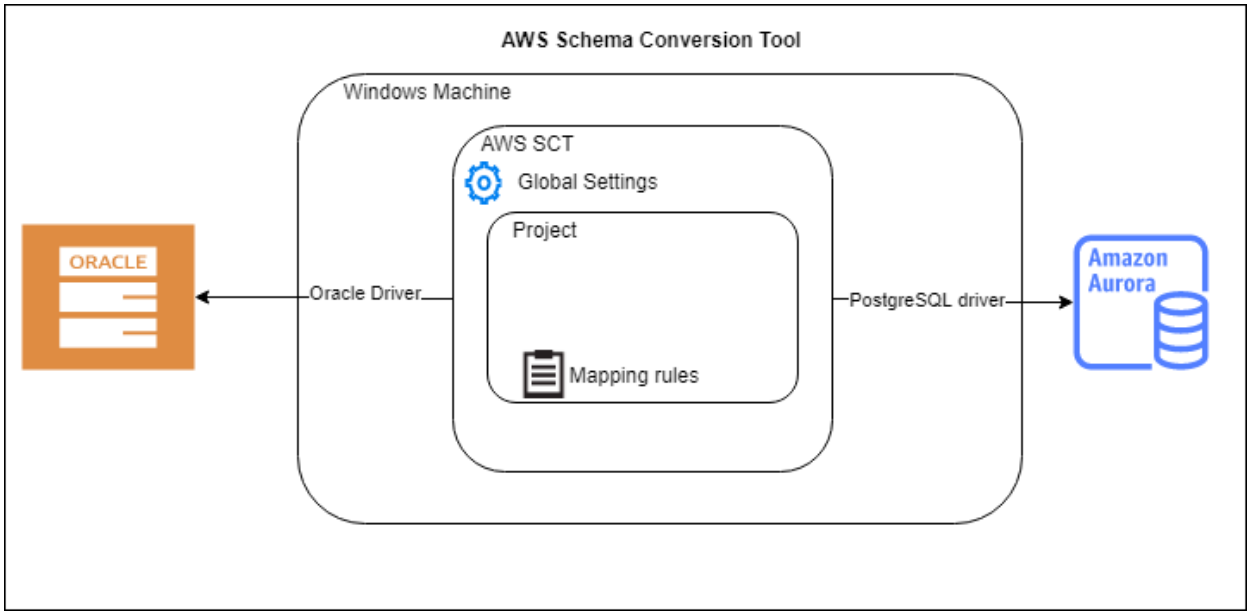
The following AWS tools and services are used to migrate the data from Oracle to PostgreSQL:

- **AWS Schema Conversion Tool** (SCT) to migrate the metadata.
- **AWS Data Migration Service** (DMS) to migrate the data.



AWS Schema Conversion Tool

AWS SCT is a standalone tool to convert the client's existing database schema from one database engine to another as a project or multiple projects. The tool converts the source database schema, and its database code objects to a format compatible with your target database, that is, PostgreSQL.



The SCT application and the supported Java Database Connectivity (JDBC) drivers on Microsoft Windows Machine are required to make the respective engine connections in order to convert the client's existing database schema.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Prerequisites

- Microsoft Windows Machine with the following specifications:
  - Microsoft Windows 10 or higher version.
  - 32GB memory
  - 2 Core CPU
  - 100GB of storage drive for SCT tasks.
- SCT Installer file
- Supported Java Database Connectivity (JDBC) drivers.

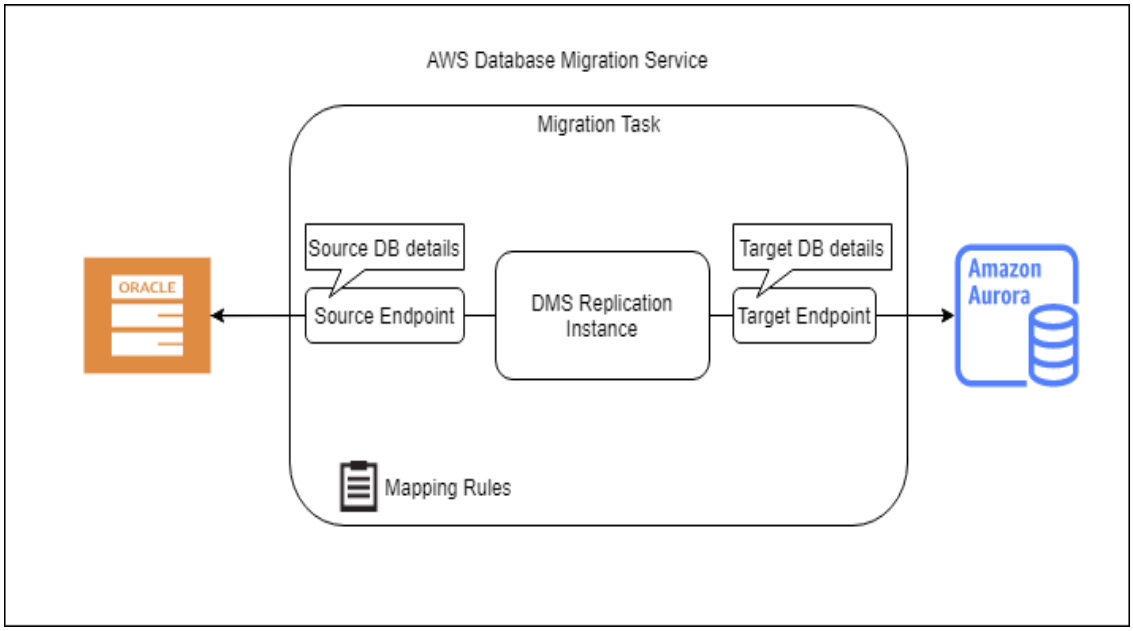
Steps performed in SCT

- Install the SCT application and the supported Java Database Connectivity (JDBC) drivers on Microsoft Windows Machine to make the respective engine connections.
- To create and manage multiple projects for different migrations between multiple database engines, configure the **Global Settings** as follows:
  - **Driver Settings**: Add the locations of the respective Oracle and PostgreSQL JDBC drivers.
  - **Performance and Memory**: Select "**Faster conversion, larger memory consumption**".
- Convert the database objects such as `Tables`, `Indexes`, `Constraints`, and `Sequences` to the target database engine format and configure the mapping rules to accomplish the following scenarios during conversion.
  - Convert `schemas`, `tables`, and `columns` to lowercase.
  - Change the data types of columns `inserted_date` and `updated_date` to timestamp from date.
  - Change datatype from number to numeric.
  - Set sequence start value to be generated from the last row count in the source database table.
- Convert the schema and extract them in SQL files.

AWS Database Migration Service

AWS DMS is a cloud service that simplifies the migration process for relational databases, data warehouses, NoSQL databases, and other types of data stores. The source database is Oracle and the target database is PostgreSQL.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.



Components of DMS

- **Endpoints:** AWS DMS uses an endpoint to access your source or target data store. Connection information of the source and target databases are entered in Endpoints.
- **DMS Replication Instance:** AWS DMS replication instance is a managed Amazon Elastic Compute Cloud (EC2) instance that hosts replication software and can host multiple replication tasks.
- **Replication Task:** AWS DMS replication task moves data from the source database to the target database. The replication tasks available are listed as follows:
  - **Full Load**: Migrates the complete data from the source database to the target database.
  - **Full Load plus Change Data Capture (CDC)**: Performs a full data load while capturing changes on the source. After the full load is complete, the captured changes are applied to the target.
  - **CDC only**: Replicates only the changes from the source database to the target database.

Migration Types

There are two approaches to Product database migration based on the database size.

- Full data migration (when the database size is less than 600 GB).
- Incremental data migration (when the database size is greater than 600 GB).

Full Data Migration

The following phases and processes involved in full data migration are as follows:

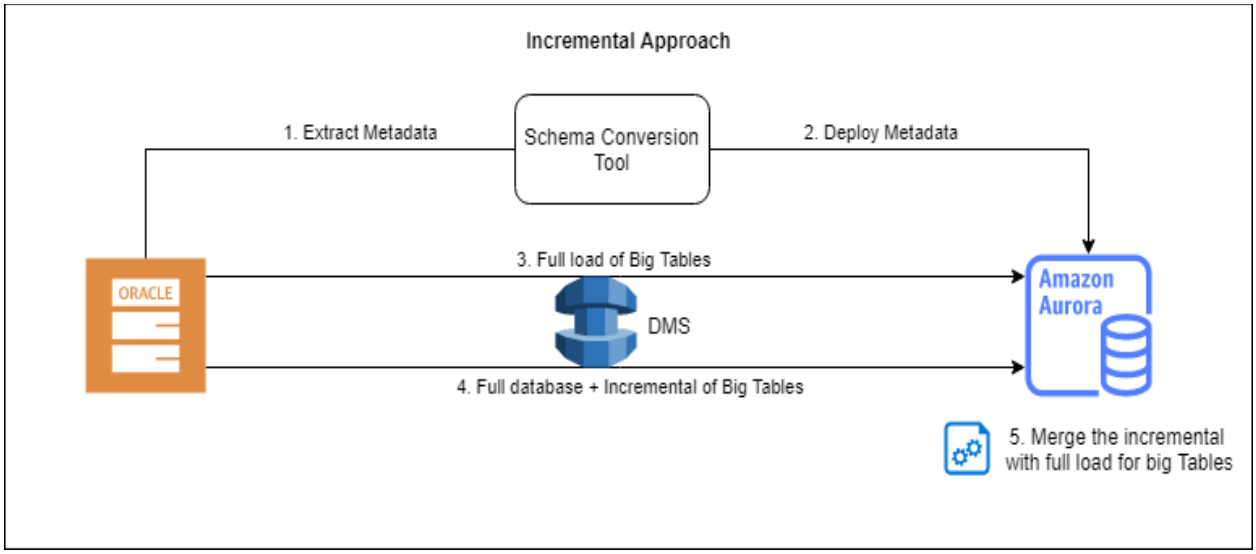
Phases	Process involved

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Pre-Migration	This phase involves the preparation of source database for the migration as follows: 1. Convert date datatype to timestamp for the required columns. 2. Create tables with the database objects information which are used to generate source and target comparison scorecards. 3. If the client's Product version on the Oracle database is V9 or lower, then scripts are processed in the Oracle staging database to get the structure compatible with V10 (this differs based on client's database size and client's downtime).
Target DB Preparation	1. Create a tablespace and empty the PostgreSQL database in target cluster. 2. Create the V3 schemas, tables structure in target database using the metadata generated from AWS SCT.
Migrate Data using DMS	1. Source and Target Endpoints: Enter the details of source and target database and test the connectivity. 2. Replication Instance: Create a DMS replication instance to migrate the data between source and target endpoints. 3. Migration Task: Migrate the data using the full load option for all the V3 schemas.
Metadata Objects	Create `Schemas`, `Tables`, `Indexes`, `Constraints`, and `Sequences` objects using the SQL files generated from AWS SCT. **Note:** `Schema` and `Tables` are created as part of the data migration. `Indexes`, `Constraints`, and `Sequences` are created post data migration.

Incremental Data Migration

The incremental approach of data migration is performed when the database size is greater than 600 GB.



The following steps are involved in the incremental data migration approach along with the full data migration process:

A week before go-live weekend:

- Get the list of big tables from the database.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

- Export big tables using the SCN number.
- Migrate big tables to target PostgreSQL database.

On the day of cutover:

- Export the full database excluding the big tables into a staging Oracle database.
- Export the incremental of big tables using the SCN number of the full export into staging Oracle database in a different schema (incremental schema).
- Migrate the Full plus incremental data to target PostgreSQL database where a full load of big tables is already in place.
- Merge the big tables using the Incremental data migrated along with the full database.
- Create `Indexes`, `Constraints`, and `Sequences` objects using the SQL files generated from SCT after the data migration process.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

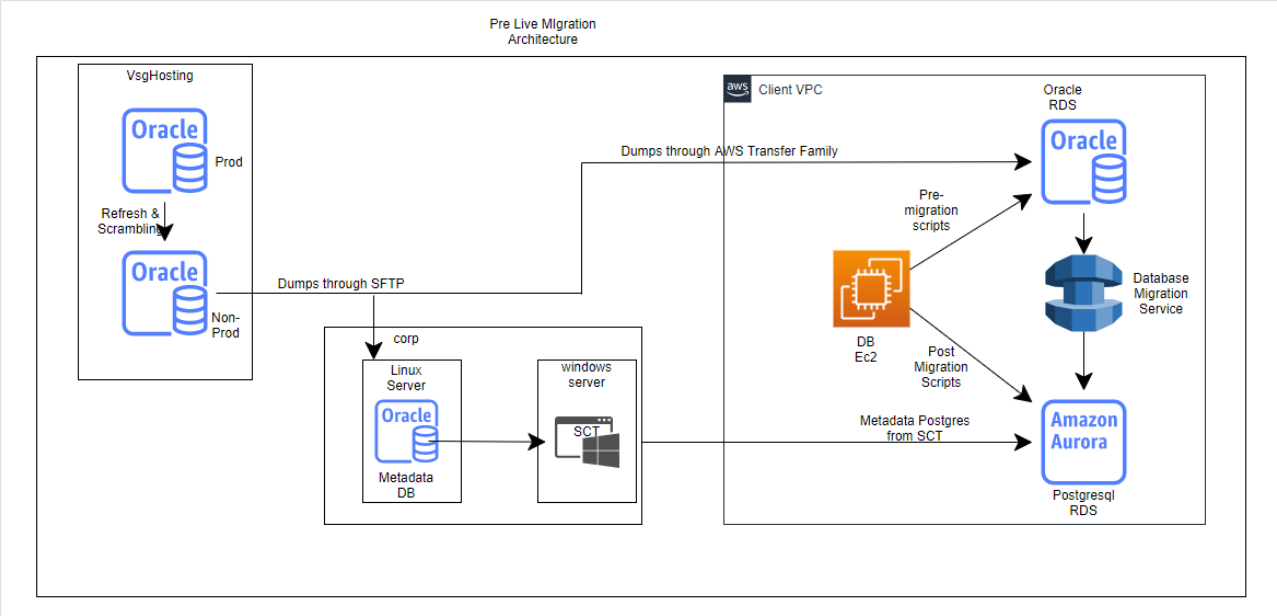
Pre-Live

Based on the data center location, the following methods are used for migrating data from Oracle to PostgreSQL for onboarding clients into Product:

- Migrating data from existing VSG Hosting data center.
- Migrating data from client on-premises.

Migration from VSG Hosting

The following illustration represents an architectural overview of the pre-live approach for data migration from VSG Hosting data center.



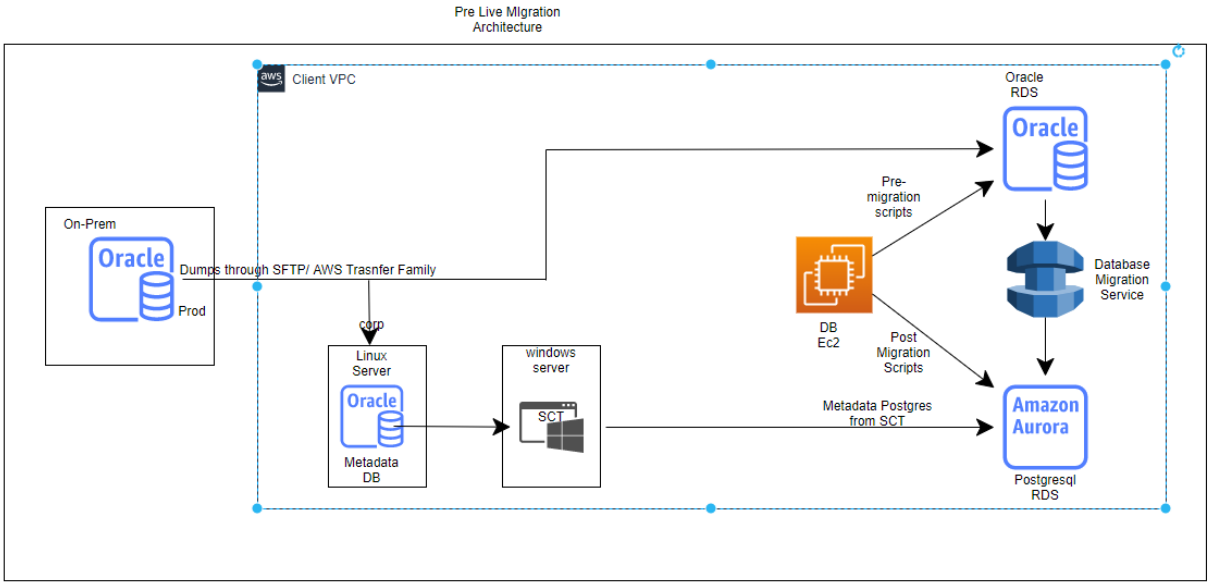
1. The data from the Oracle Production environment is refreshed to a Non-Production environment and is scrambled to remove any PII (Personally Identifiable Information) data.
2. Once the scrambling is completed, database dumps are exported with the help of Oracle Data Pump Export utility using `expdp` command. Both the metadata and full database dumps are exported separately.
3. The metadata dump is then transferred to an empty Oracle database in the Corp environment which is used as a source for AWS SCT.
4. Full database dump is compressed and transferred from VSG Hosting to Client VPC in Product using AWS Transfer Family Service. Dumps are then imported into staging Oracle RDS which acts as a source for AWS DMS.
5. In preparation for migration, a set of pre-migration SQL queries are run on both metadata Oracle database in corp and staging Oracle RDS in client VPC. These SQL queries include tables for migration scorecard, changes in Oracle datatypes to PostgreSQL supported datatypes, and metadata changes required for V3 Upgrade from version 9 or lower to version 10.
6. Once the pre-migration scripts are run on both Oracle instances, SCT is run to generate metadata for the target Aurora PostgreSQL instance. This metadata (table structure) is then moved to the target Aurora PostgreSQL instance to be available for DMS.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

7. DMS is run against source Oracle RDS and the target Aurora PostgreSQL RDS to migrate the data. Once the data is migrated, post-migration scripts are run against the target which includes constraints, indexes. The migrated data is handed over to the Development Team for the Code migration and Upgrade process.

Migration from On-Premises

The following illustration represents an architectural overview of the pre-live approach for data migration from on-premises.



1. The scrambled metadata dump and full database dump are provided by the client through SFTP/AWS transfer family setup.
   a. The metadata dump is imported into the metadata Oracle database in the Linux server and
   b. The full database dump is imported in Oracle staging RDS.
2. The subsequent steps are same as the migration process from VSG Hosting. The migrated data is handed over to the Development Team for the Code migration and Upgrade process.

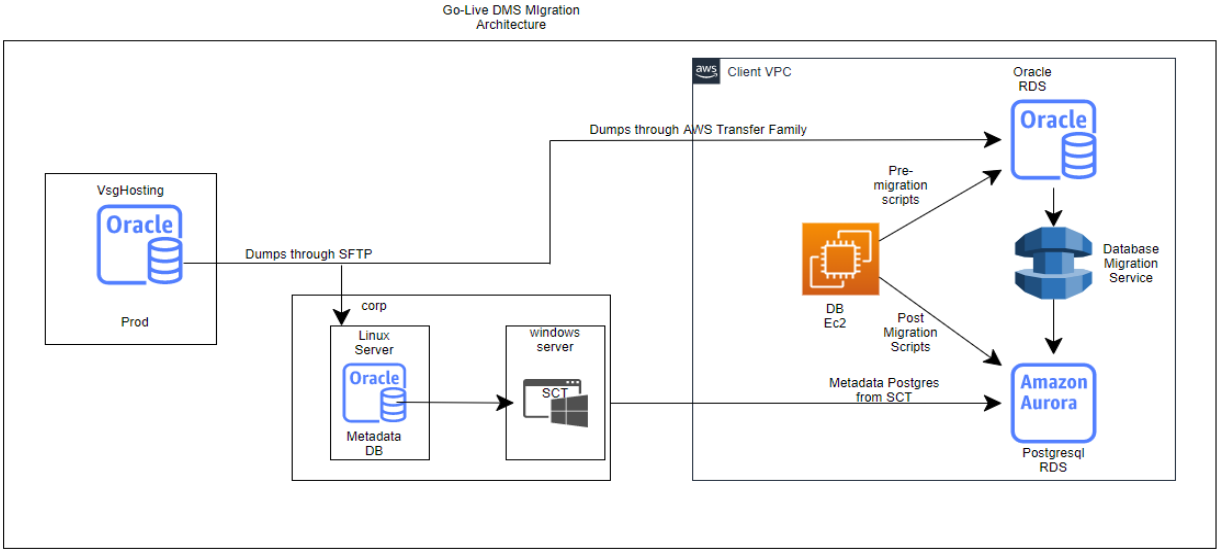
© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Go Live

The Go-Live migration is performed to accommodate migration for large-size databases (greater than 600 GB). During Go-Live, large tables are migrated ahead of the go-live weekend and incremental data is migrated on go-live weekend.

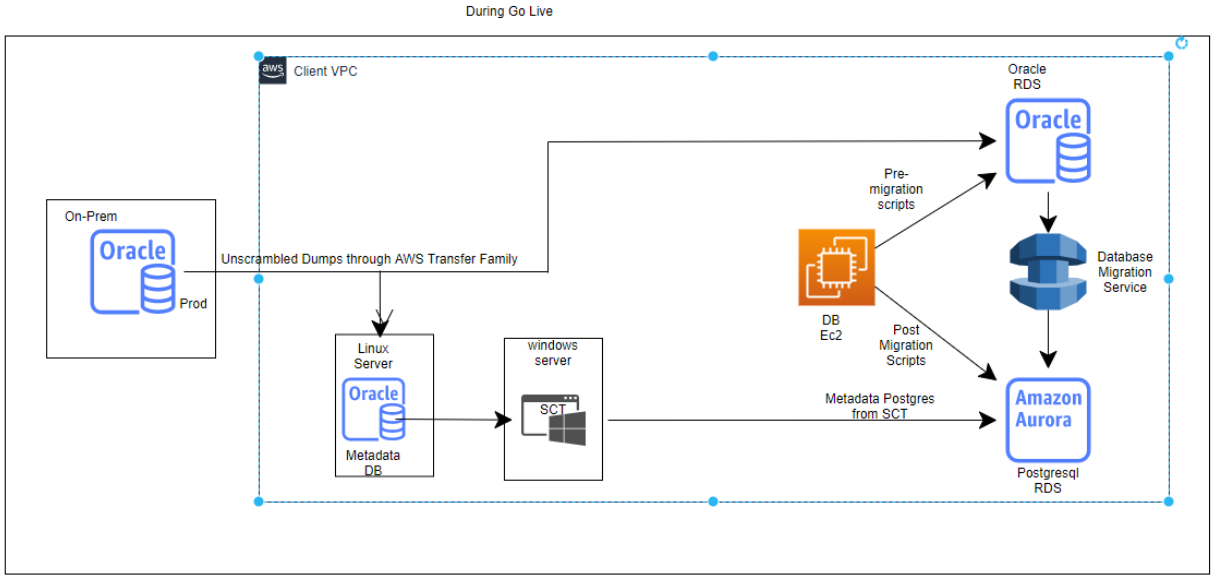
Migration from VSG Hosting

During Go-Live, unscrambled export dumps are extracted directly from the Production database instead of the lower environment. The subsequent steps are same as the migration process from the Pre-Live approach.



Migration from On-Premises

The unscrambled data dumps are extracted directly from Production environment and provided by the client. The subsequent steps are same as the migration process from Pre-Live approach.



© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Report Generation

A comparison report is generated after migration, which displays a summary of the differences from pre and post-migrated data which can be used to validate the migrated data. The report displays differences in tables, tables row count, indexes, constraints, default value type for table columns, and datatype in table columns.

The report generation process includes the following steps:

- Create tables to hold the database objects information, along with the row count in the tables after the migration.
- Create views that compare the source and target database objects.
- Run the Python script after the database migration. This will generate the comparison report in excel format, a sample report is shown below.

	A	B	C	D	E	F	G	H	I
1	owner	table_name	ora_count	pg_count	difference				
2	BENDEV	ALL_TABLES_ROW_COUNT	59	3772	-3713				
3	DBO	DR$BEK_SEARCH_01_IDX_1$I	3052639	0	3052639				
4	DBO	DR$BEK_SEARCH_01_IDX_1$K	1282792	0	1282792				
5	DBO	DR$BEK_SEARCH_01_IDX_1$N	815	0	815				
6	DBO	DR$ENC_SEARCH_01_IDX_1$I	113	0	113				
7	DBO	DR$ENC_SEARCH_01_IDX_1$K	70	0	70				
8	DBO	DR$IDX_DOCUMENT$K	23	0	23				

Sample report showing table row-count difference

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Chapter 3 – Code Migration

As a part of Product onboarding, code migration (Oracle to PostgreSQL) is performed on the code that resides in the client SVN. The code and its associated objects from client Oracle SVN are converted to PostgreSQL compatible format using 's inbuilt tool.

The code migration process is carried out in two stages:

Oracle to PostgreSQL

Java to PostgreSQL

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

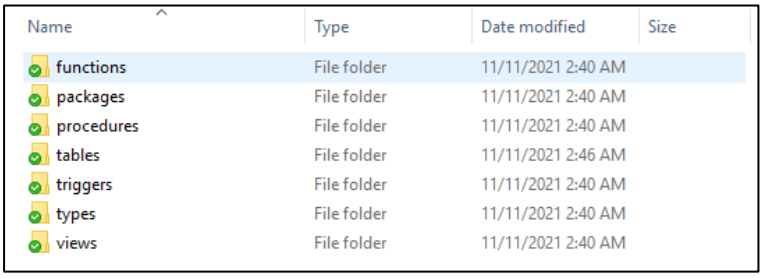
Code Migration - Oracle to Postgres Database

During Product onboarding, client Oracle code has to be converted into a Postgres supported format. The Oracle to Postgres code migration process followed by is as follows.

Prerequisites for Clients

Before the Oracle to Postgres code migration, as a prerequisite, the clients must do the following:

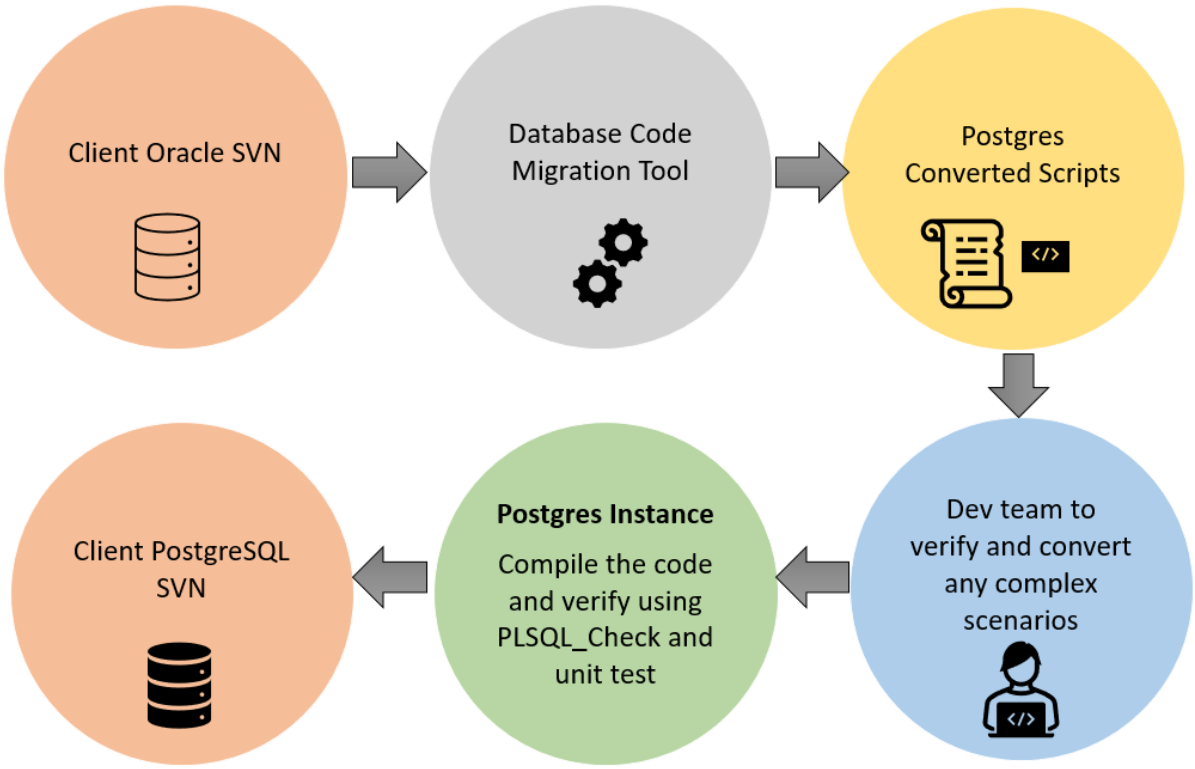
- Clean up any duplicate/redundant objects and move valid APIs from `DML/DDL` exception folder to respective `function/procedure/package/triggers/views/types` folder.
- Client objects that are available in the source database must also be maintained in the Client SVN in the same folder structure.
- `DML/DDL Exceptions` folders will not be considered for conversion.
- The recommended Client SVN folder Structure for Oracle appears as follows.



Name	Type	Date modified	Size
functions	File folder	11/11/2021 2:40 AM	
packages	File folder	11/11/2021 2:40 AM	
procedures	File folder	11/11/2021 2:40 AM	
tables	File folder	11/11/2021 2:46 AM	
triggers	File folder	11/11/2021 2:40 AM	
types	File folder	11/11/2021 2:40 AM	
views	File folder	11/11/2021 2:40 AM	

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

How the Oracle to Postgres Code Conversion Process Works



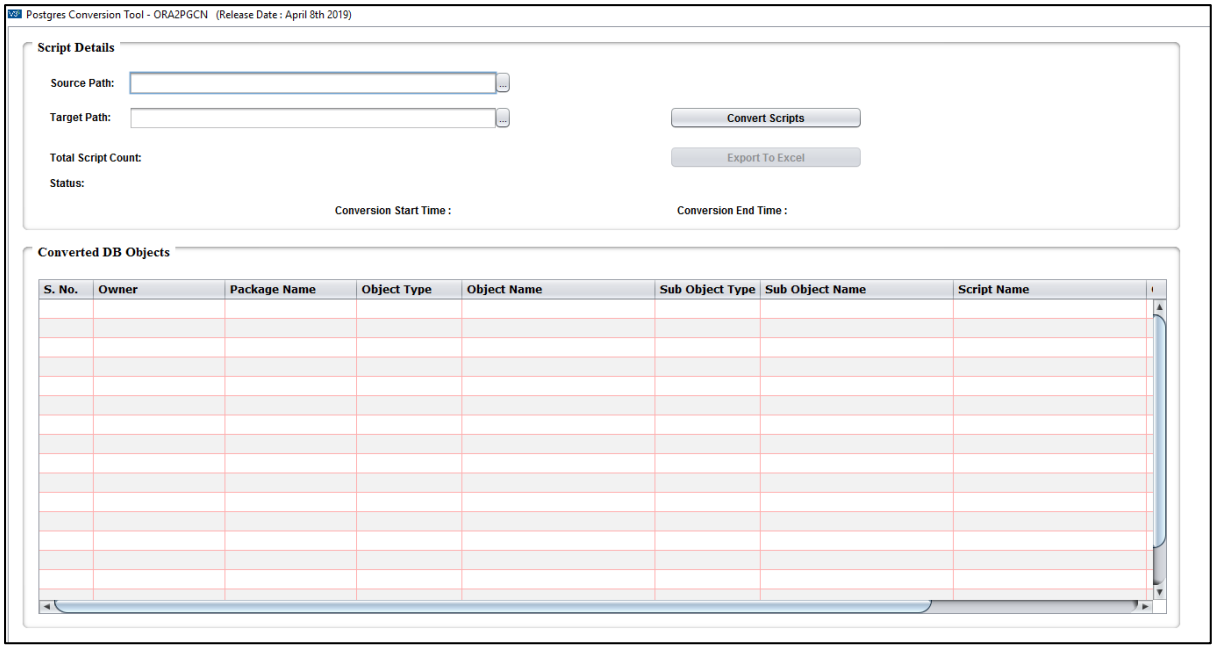
- The Code Migration process is performed using an internal tool, that automates all the repetitive and manual tasks to convert Oracle code to PostgreSQL compatible code. Based on the complexity involved, the tool converts about 60% of the code.
- The tool accepts source and target path. The source path stores the location of the script objects to be converted (Client Oracle SVN path). After converting the script objects, the files will be stored in the Target path in a similar folder structure as the source. A report is generated to validate the conversion.
- Post the automated process, the Developers from the migration team converts the pending/failed code manually along with views by compiling it in Postgres instance. The `pgpsql_check` extension is also used to find any compilation errors.
- A unit testing is carried out and after successful validation of code conversion, the converted code is checked in to the client's Postgres SVN with the same file naming convention as Oracle SVN.
- Along with code objects conversion, the Oracle database compatible queries are also updated (for example, business rule expression value for expression type as **sql**) to PostgreSQL compatible queries which are used from JAVA.

Note: This step is a one-time activity to convert all the client database code objects from Oracle to Postgres using a combination of the automated tool and manual effort, depending on the complexity of code objects.

Postgres Conversion Tool

Tool Location: `S:\ORA_PG_Tool\PGConversionTool.jar`

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.



During code conversion, the migration team follows best practices and query optimization techniques, such as reducing unnecessary type casting and using built-in functions, to improve performance in PostgreSQL.

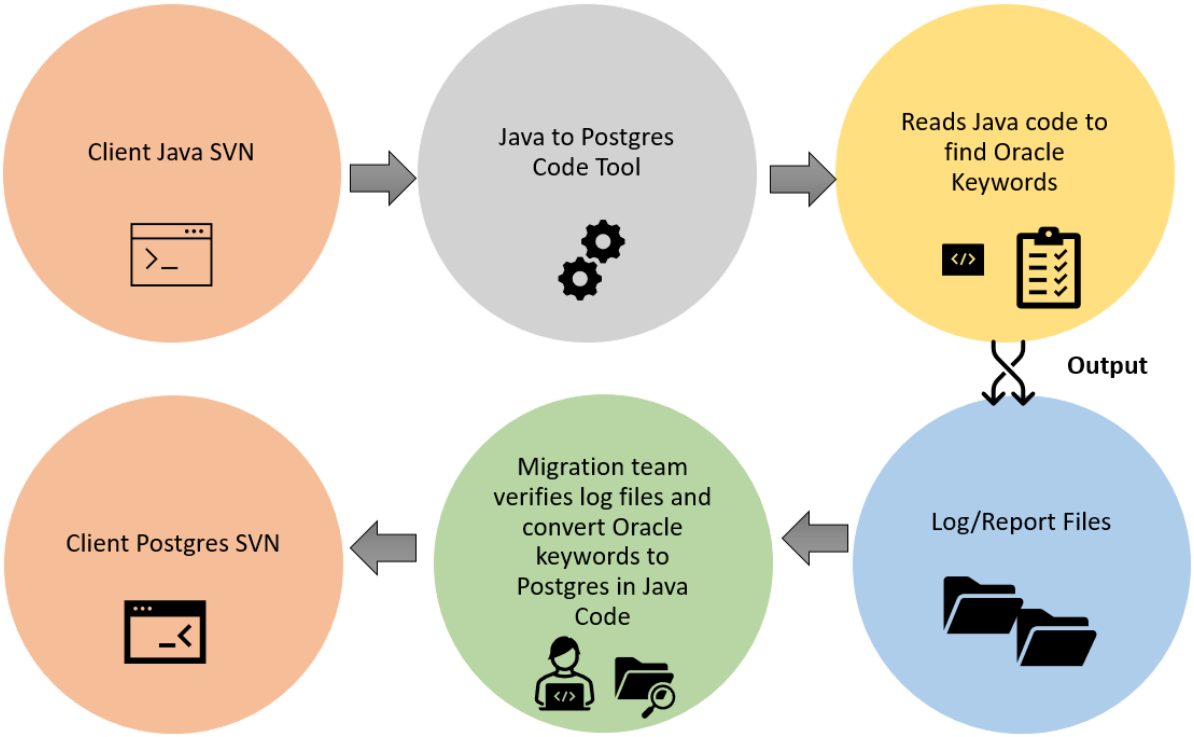
For more information about the best practices and performance improvement techniques, see the following links:

Best Practices of PostgreSQL
Performance Improvement in PostgreSQL

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.

Code Migration – Java to Postgres Database

’s automated Java to Postgress Code tool converts the Java code and its associated objects from the client SVN to PostgreSQL.



The tool reads the Java code and its associated objects to find Oracle keywords and converts the code into Postgres compatible format. The tool generates a log/report file with the summary of conversion activities for verification. The Migration team verifies the log files for conversion and further converts any pending Oracle keywords to Postgress supported format. The converted code is then moved to the Client Postgres SVN.

© 2026 Akash Raj. All rights reserved.
No part of this work may be reproduced, distributed, or used without prior permission.